

Help, programming is hard!

1. Prerequisites

If you miss solving homework in Python and Java, and don't feel quite ready for another Haskell semester, these tips are for you!

Try them out. They can make you a sharper programmer (and mathematician!) even when Haskell is not around.

2. Interface and implementation

An **interface** (contract, promise, problem, ...) is a description of what something should behave like from the outside.

Example 2.1: Here is an interface for a jukebox.

```
1 If a jukebox is not plugged in, it does nothing. If a jukebox is plugged in, it
2 accepts coins. If a coin is inserted in a plugged-in jukebox, a music track can
3 be selected from the menu. Said track will be played once after stopping any
4 other track that was being played.
```

An **implementation** (internals, solution, ...) of an interface $\mathcal{I}$ is an object being correctly described by $\mathcal{I}$. If you go to a jukebox museum, you will see plenty of implementations of the same jukebox interface given above.

Do you remember some reasons why this distinction is useful?

3. Making an interface rigorous and finding an implementation

When descriptions of what something should do are expressed in natural language, they might make for very ambiguous interfaces. We first need to disambiguate these descriptions, so that they become **rigorous**.

Example 3.1: Here are some of the ambiguities in the interface given above:

```
1 What counts as a "coin"? What doesn't?
2 What happens if a coin is inserted, but no song is selected?
3 Can just any amount of time pass between the previous track being stopped and
4 the next one being played?
```

4. Types are interfaces. Expressions are their implementations

Types are descriptions of what expressions should behave like, so they can work as interfaces whose implementations are terms.¹ Types in Haskell are pretty unambiguous.

Example 4.1:

```
1 type Chooser a = a -> a -> a
```

If a is a description (type), `Chooser a` describes all Haskell binary functions whose inputs and outputs both match description a . The compiler will check if our proposed solution correctly implements the interface it is trying to implement.

¹Some people might argue that only code starting with Java's `interface` keyword is a real interface, but that's a conversation for another time.

Sometimes, our programming language is too limited to express all parts of our description as types, so we have to resort to comments.

Example 4.2:

```
1 -- For all types a, f :: Chooser a, x :: a, y :: a, the following must hold:
2 -- f x y = x or f x y = y
3 type Chooser a = a -> a -> a
```

Both types and comments are documentation tools, but:

- the compiler can only check that our proposed solution is correct with reference to its type, and not against the comments;
- our code editor, which is trying to help us code, can have tools to perfectly understand the types. Alas, tools that try to guess what our natural language comments mean might always make a mess.

5. Type-driven development and hole programming

Can we use **types** to make less mistakes and understand the problem better?

Holes (underscores: `_`) can be used as placeholders for expressions you haven't come up with yet. They allow you to move on with your coding without your code editor getting confused at your partial solution.

When implementing a program, try to:

1. write its **interface** as a type+comment;
2. as you try to write an **implementation**, if you get stuck:
 1. use a hole as a temporary function body;
 2. think about the interfaces you need, and implement those recursively invoking this procedure;
 3. come back to the hole and fill it.

Example 5.1: We can implement the `reverse` function from Haskell's prelude using type-driven development and hole programming. (See live coding session in class.)

6. Searching by type

Types can even help you outside your code editor. The following search engine (which you can install for offline use) works even if you don't know the name of what you are looking for: hoogle.haskell.org. Just type the desired type signature in the search box. You can use the dropdown menu to filter by specific package.

7. Is my code correct? I don't want to lose points...

When doing maths, we can afford to work with proofs because we have rigorously defined the objects we are dealing with. However, we don't have the time to rigorously define what each Haskell program does, so we will **almost never ask you to prove** your Haskell program correct. How do we gain (some) confidence on program correctness?

- If you typed your function correctly (did you?), and are not using tricks to cheat the type systems, your code compiling is at least proof that your program respects the interface given by the **types** (but not necessarily the one given by the comments);

- If you constructed your Haskell code so that it looks uncannily similar to the definitions of the mathematical objects you are asked to implement, you could say it is **correct by construction**. However, as stated above, we haven't rigorously defined what each Haskell program does, code that looks very similar to what's in the lecture notes might actually be subtly different;
- You could test each of your definitions on some **examples**. However, examples are NOT proofs!